

C++ Win32 Console 기반 자체 엔진 게임 프로젝트

구현 방법 · 진행 일정 · 트러블슈팅 · 회고

김지형 (Kim Jihyung) | 2025. 8. 11 ~ 8. 15

프로젝트 개요

CraftEngine (DLL)

Win32 Console API 위에 직접 구축한 자체 엔진
메인 루프 · 렌더러 · 액터 · 충돌 · 입력 · 카메라
게임 도메인을 전혀 모르는 순수 엔진 계층

ShootingGame (EXE)

엔진 위에서 동작하는 실제 게임
마을(소코반 RPG) + 던전(뱀파이어 서바이버)
퀘스트 · 대화 · NPC · 세이브 · 웨이브

88

소스 파일
(.h / .cpp)

5

레벨
(Menu~Survivor)

14

데이터 파일
(맵/퀘스트/웨이브)

5일

개발 기간
(8/11 ~ 8/15)

구현 방법 ① 엔진 코어

1

고정 프레임 메인 루프

QueryPerformanceCounter 고해상도 타이머로 120FPS 고정.
Input → BeginPlay → Tick → Collision → Draw → 추가/삭제 처리 순서

2

이중 버퍼링 렌더링

CreateConsoleScreenBuffer로 버퍼 2개 생성 → CHAR_INFO 배열에 합성 후
WriteConsoleOutputW로 한 번에 출력 → SetConsoleActiveScreenBuffer 토글

3

뷰포트 분할

RenderSpace enum(Game/UI/Dialogue/Full)으로 화면 4분할.
Setting.txt에서 각 뷰포트 크기를 읽어 재컴파일 없이 레이아웃 변경

4

카메라 (맵 > 화면)

offset = 플레이어 - 뷰포트/2, clamp로 맵 경계 제한.
Submit 시 Game 공간만 offset 차감 → UI는 고정(HUD)

구현 방법 ② 게임플레이 시스템

SpatialGrid

공간 분할 충돌

맵을 5×5 셀로 분할.
Insert $O(1)$ / Query는 자기 셀 +
인접 8셀만 검사.
 $O(N^2) \rightarrow$ 실질 $O(N)$

ActorPool<T>

오브젝트 풀링

총알 200 / 적 150 사전 할당.
Acquire=Activate,
Release=Deactivate.
런타임 힙 할당 0회

EnemyFSM

유한 상태 머신

Idle $\rightarrow$ Chase $\rightarrow$ Rush 3상태.
전이 조건은 맨해튼 거리.
임계값·배율만 바뀌
변종 적 생성 가능

WaveManager

난이도 곡선

WaveConfig.txt에서
시간·적 타입·HP·속도·보상 파싱.
파일 없으면 기본 테이블 폴백

구현 방법 ③ 콘텐츠 · 데이터

공통 파싱 파이프라인

1. `DataFileUtil::ReadAllLines()`
2. `SplitKeyValue()` → "KEY: VALUE"
3. `Trim()` → Windows CR(\r) 제거
4. 각 Manager가 자기 구조체로 변환

파서 하나를 4종 데이터가 공유
→ 새 데이터 타입 추가 시 파서 재사용

Setting.txt까지 데이터화하여
FPS · 뷰포트 크기 · 마진을 재컴파일 없이 조절

QuestManager

Q1~Q3.txt · KILL / SOKOBAN / DIALOGUE 3종
NotStarted → InProgress → Completed 상태 전이

DialogueManager

0.05초마다 UTF-8 1글자씩 타이핑 연출
완료 콜백으로 퀘스트 수락·보상 연결

NPCManager

NPCs.txt에서 이름·초상화·대사·퀘스트 매핑
맵의 'N' 문자에 순서대로 npcId 부여

SaveManager

스탯 + 퀘스트 진행도 + 맵 상태(플레이어/박스)
텍스트 직렬화, 로드 플래그로 1회만 적용

프로젝트 진행 일정 (5일)

1일차

8/11

뷰포트 분할 · 대화/퀘스트 골격

버퍼를 Game/UI로 분할 (RenderSpace enum) · DrawBorderOutline 외곽선
DialogueSystem · QuestSystem 선언 · 화면보다 큰 맵을 위한 Camera 클래스 추가

완료

2일차

8/12

CanMove 최적화 · 상태 분리

CanMove() 리팩토링 — 비트연산 키 + 해시테이블로 O(1) 판정
Dialogue 뷰포트 추가 · UIManager · 레벨 이동해도 유지되는 PlayerState

완료

3일차

8/13

한글 출력 · 충돌 터널링 해결

MultiByte/UTF-8 → WideChar 변환으로 한글 출력 · ConfirmUI 선택지
QuestSystem 세부 구현 · DataFileUtil 네임스페이스로 파싱 공통화

완료

4일차

8/14

SurvivorLevel · 성능 최적화

맵서형 던전 추가 — EnemyFSM · ActorPool · SpatialGrid · WaveManager
Quest/Dialogue/NPC 전부 txt 데이터화 · 골드 재화로 스탯 강화 구현

완료

5일차

8/15

세이브/로드 · 발표 준비

SaveManager 구축 — 스탯/퀘스트/맵 상태 직렬화 · MenuLevel Load 연동
잔여 버그 픽스 (한글 겹침 · 타이핑 바이트 깨짐) · 발표 자료 및 시연 시나리오

완료

트러블슈팅 ① 한글 출력

UTF-8 한글은 3바이트, 콘솔에서는 2칸을 차지한다

문제 글자 겹침 — 대사가 깨져서 출력

원인

렌더러가 "1 wchar_t = 1칸"으로 가정.
'안'을 셀[0]에 넣고 '녕'을 바로 셀[1]에 기록 →
'안'이 이미 쓰고 있던 셀[1]을 덮어씀

해결

IsWideChar()로 유니코드 코드포인트 범위 판별.
전각이면 col += 2로 전진

문제 타이핑 연출에서 글자 깨짐

원인

currentLine[revealedCount]로 바이트 단위 접근.
한글 3바이트를 1바이트씩 잘라 붙이니
중간 상태가 전부 깨진 문자로 출력

해결

UTF-8 선두 바이트를 분석해 해당 문자의 길이(1~4B)를
구한 뒤 substr로 한 글자 통째로 추출.
revealedCount도 바이트 단위로 함께 전진

트러블슈팅 ② 이동 · 충돌

문제 등속 이동 시 플레이어가 벽을 뛰어넘음 (터널링)

원인

소코반은 GetKeyDown이라 한 칸씩 이동 → 충돌 정상
하지만 GetKey 등속 이동으로 바꾸니 한 프레임에
여러 칸을 건너뛰며 충돌 판정 후에도 위치가 갱신됨
ex) (1,1) → 목표 (4,1) 이면 중간 (2,1) (3,1)을 건너뛴

해결

이전 위치와 목표 위치가 다르면 그 사이를 한 칸씩 순회하며
단계별로 CanMove() 검사
막히는 칸을 만나면 그 직전 칸까지만 이동하고
실수 좌표(floatPos)까지 롤백 → 다음 프레임 재가속 방지

문제 엔진 Swept AABB의 오탐 — 스쳐 지나가도 피격 판정

원인

엔진 충돌은 이전 위치 ~ 현재 위치를 합산한
확장 바운딩 박스를 씌 (고속 물체 터널링 방지 목적)
던전에서는 이게 독 — 빠른 적이 옆을 스쳐만 가도
"경로 전체"가 충돌 영역이라 체력이 깎임

해결

엔진은 DLL이라 CollisionSystem 수정 불가 → 우회 설계
① Tick()에서 자체 충돌을 먼저 실행 (현재 위치 기준)
→ 처리된 액터를 Deactivate()
② 적/플레이어의 OnCollision()을 의도적으로 비움
→ 엔진이 오탐을 알려도 아무 일도 일어나지 않음

결과 : 모든 충돌 로직이 ProcessGridCollision() 한 곳에 집중 → 오탐 제거 + 디버깅 지점 단일화

트러블슈팅 ③ 성능 병목

CanMove() 중복 순회

BEFORE

장애물 종류마다 actorList를 각각 순회
한 번 이동할 때 수백 개 액터를 여러 바퀴 검색

AFTER

정적 타일은 맵 로딩 시 해시맵에 사전 등록
EncodeTilePos(x,y) : y를 상위 32bit,
x를 하위 32bit에 넣어 int64 키 생성
tileMap.find(key) 한 번으로 O(1) 판정

움직이는 박스만 별도 리스트 선형 탐색 (4~5개)

충돌 검사 $O(N^2)$ 전수 비교

BEFORE

적 50 + 총알 200 = 250 액터
→ $250 \times 249 / 2 = 31,125$ 회 비교 / 프레임

AFTER

맵을 5×5 셀로 분할 (SpatialGrid)
총알 위치의 셀 + 인접 8셀만 조회
셀당 평균 적 2~3마리 (K)
→ $200 \times 3 =$ 약 600회 수준으로 감소

셀 크기 5 = 충돌 가능 최대 폭(4칸)보다 크게 설정

공통 원칙 : "매번 전부 뒤지지 말고, 미리 정리해두거나 후보를 좁힌다"

리팩토링 Before / After 총정리

항목	Before	After
이동 판정	actorList를 장애물마다 반복 순회	비트연산 키 + 해시맵 O(1) 조회
벽 관통	등속 이동 시 여러 칸 건너뛰며 통과	경로를 한 칸씩 단계 검사 후 롤백
충돌 검사	$O(N^2)$ 전수 비교 · 31,125회/프레임	SpatialGrid 인접 9셀 · 약 600회
충돌 오탐	Swept AABB가 경로 전체를 영역으로 판정	현재 위치만 비교 · OnCollision() 비율
총알/적 생성	매 발사마다 힙 할당 / 해제 반복	ActorPool 사전 할당 후 Activate 재활용
적 AI	if/else 단일 추적 행동	EnemyFSM 3-State · 전이 조건 명시
난이도 관리	코드 하드코딩 · 수정마다 재컴파일	WaveConfig.txt 데이터 드리븐
한글 렌더링	1 wchar_t = 1칸 가정 · 글자 겹침	IsWideChar + LEADING/TRAILING 속성
대사 타이핑	바이트 단위 접근 · 한글 3바이트 깨짐	UTF-8 선두 바이트 분석 후 글자 단위 추출
텍스트 정렬	string::length()로 폭 계산 · 외곽선 밀림	GetDisplayWidth()로 실제 표시 폭 산출

회고 ① 아쉬운 점 · 보완할 점

X 로컬라이징 미완

key → 현지 언어 치환 테이블을 설계만 하고
결국 한국어 하드코딩. 정렬도 한글 기준 고정

X 콘솔 리사이즈 대응

Ctrl+휠로 창 크기를 바꾸면 폰트/셀 크기가 변해
외곽선이 흩어짐. 근본 대응 없이 넘어감

X 사운드 · 폴리싱 부재

타격감, 피격 연출, BGM이 전혀 없음.
체감 완성도를 크게 떨어뜨리는 부분

X 최적화 범위의 불균형

SpatialGrid를 던전에만 적용.
마을은 액터가 적다는 이유로 후순위로 미룸

→ StringTable 도입

모든 출력 문자열을 key로 치환하고
String_ko.txt / String_en.txt 분리 로드

→ 리사이즈 이벤트 처리

콘솔 창 크기 변경을 감지해
뷰포트를 재계산하고 프레임 버퍼 재생성

→ 연출 레이어 추가

히트스톱, 화면 흔들림, 데미지 숫자 등
이미 있는 DestroyEffect 구조를 확장

→ SpatialGrid 엔진 이관

게임 코드가 아닌 엔진 계층으로 옮겨
모든 레벨이 공통으로 사용하도록 일반화

가장 큰 배움 : "돌아가게 만드는 것"과 "유지보수 가능하게 만드는 것"은 다른 작업이다

Thank You

김지형 (Kim Jihyung)